LAPORAN PRAKTIKUM

STRUKTUR DATA

MODUL 5

SINGLY LINKED LIST (BAGIAN KEDUA)



Disusun oleh:

Itrhoh Anggun Pamungkas

109082500117

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul 5 :
https://github.com/anggungun/struktur_data/tree/main/Modul5

GUIDE

1. **Guide 1**

   SinglyLinked.h

```
#ifndef SLL_H
#define SLL_H

struct ElmList {
    int info;
    ElmList* next;
};
struct List {
    ElmList* First;
};

// Fungsi dasar
void CreateList(List &L);
ElmList* alokasi(int nilai); //alokasi untuk membuat node baru
dan mengembalikan alamat node tersebut
void dealokasi(ElmList* &P);
void printInfo(List L);

// Searching
ElmList* findElm(List L, int nilai);

// Insert
void insertFirst(List &L, int nilai);
void insertLast(List &L, int nilai);
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru);

// Delete
void delFirst(List &L);
void delLast(List &L);
void delAfter(List &L, int nilaiSebelum);
void delP(List &L, int nilai);

// Operasi tambahan
void update(List &L, int nilaiLama, int nilaiBaru);
int nbList(List L);
void delAll(List &L);

#endif
```

SinglyLinked.cpp

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

// Membuat list kosong
void CreateList(List &L) {
    L.First = nullptr;
}

// Membuat node baru
ElmList* alokasi(int nilai) {
    ElmList* P = new ElmList;

    P->info = nilai;
    P->next = nullptr;

    return P;
}

// Membebaskan memori node
void dealokasi(ElmList* &P) {
    delete P;
    P = nullptr;
}

// Menampilkan semua isi list
void printInfo(List L) {
    ElmList* P = L.First;

    while (P != nullptr) {
        cout << P->info << " ";
        P = P->next;
    }

    cout << endl;
}

// Mencari node berdasarkan nilai
ElmList* findElm(List L, int nilai) {
    ElmList* P = L.First;

    while (P != nullptr) {
        if (P->info == nilai) {
            return P;
        }
```

```
        P = P->next;
    }

    return nullptr;
}

// Menambahkan node di awal
void insertFirst(List &L, int nilai) {
    ElmList* P = alokasi(nilai);

    P->next = L.First;
    L.First = P;
}

// Menambahkan node di akhir
void insertLast(List &L, int nilai) {
    ElmList* P = alokasi(nilai);

    if (L.First == nullptr) {
        L.First = P;
    } else {
        ElmList* Q = L.First;

        while (Q->next != nullptr) {
            Q = Q->next;
        }

        Q->next = P;
    }
}

// Menambahkan node setelah nilai tertentu
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru) {
    ElmList* Q = findElm(L, nilaiSebelum);

    if (Q == nullptr) {
        cout << "Nilai sebelumnya tidak ditemukan." << endl;
    } else {
        ElmList* P = alokasi(nilaiBaru);

        P->next = Q->next;
        Q->next = P;
    }
}

// Menghapus node pertama
void delFirst(List &L) {
    if (L.First == nullptr) {
```

```
        cout << "List kosong." << endl;
    } else {
        ElmList* P = L.First;

        L.First = L.First->next;
        dealokasi(P);
    }
}

// Menghapus node terakhir
void delLast(List &L) {
    if (L.First == nullptr) {
        cout << "List kosong." << endl;
    } else if (L.First->next == nullptr) {
        delFirst(L);
    } else {
        ElmList* P = L.First;

        while (P->next->next != nullptr) {
            P = P->next;
        }

        ElmList* Q = P->next;
        P->next = nullptr;
        dealokasi(Q);
    }
}

// Menghapus node setelah nilai tertentu
void delAfter(List &L, int nilaiSebelum) {
    ElmList* P = findElm(L, nilaiSebelum);

    if (P == nullptr) {
        cout << "Nilai sebelumnya tidak ditemukan." << endl;
    } else if (P->next == nullptr) {
        cout << "Tidak ada node setelah nilai tersebut." <<
endl;
    } else {
        ElmList* Q = P->next;

        P->next = Q->next;
        dealokasi(Q);
    }
}

// Menghapus node berdasarkan nilai
void delP(List &L, int nilai) {
    if (L.First == nullptr) {
```

```
        cout << "List kosong." << endl;
    } else if (L.First->info == nilai) {
        delFirst(L);
    } else {
        ElmList* P = L.First;

        while (P->next != nullptr &&
               P->next->info != nilai) {
            P = P->next;
        }

        if (P->next == nullptr) {
            cout << "Nilai tidak ditemukan." << endl;
        } else {
            ElmList* Q = P->next;

            P->next = Q->next;
            dealokasi(Q);
        }
    }
}

// Mengubah nilai data
void update(List &L, int nilaiLama, int nilaiBaru) {
    ElmList* P = findElm(L, nilaiLama);

    if (P == nullptr) {
        cout << "Nilai tidak ditemukan." << endl;
    } else {
        P->info = nilaiBaru;
    }
}

// Menghitung jumlah node
int nbList(List L) {
    int jumlah = 0;
    ElmList* P = L.First;

    while (P != nullptr) {
        jumlah++;
        P = P->next;
    }

    return jumlah;
}

// Menghapus semua node
void delAll(List &L) {
```

```
    while (L.First != nullptr) {
        delFirst(L);
    }
}
```

main.cpp

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

int main() {
    List L;
    ElmList* hasil;
    ElmList* P;
    int total = 0;

    CreateList(L);

    // 1. Insert First
    insertFirst(L, 2);
    insertFirst(L, 0);
    insertFirst(L, 8);
    insertFirst(L, 12);
    insertFirst(L, 9);

    cout << "List awal: ";
    printInfo(L);

    // 2. Searching
    hasil = findElm(L, 8);
    if (hasil != nullptr) {
        cout << "Angka 8 ditemukan." << endl;
    } else {
        cout << "Angka 8 tidak ditemukan." << endl;
    }

    // 3. Menghitung total nilai
    P = L.First;
    while (P != nullptr) {
        total = total + P->info;
        P = P->next;
    }
    cout << "Total nilai: " << total << endl;

    // 4. Insert Last
    insertLast(L, 20);
    cout << "Setelah insertLast(20): ";
    printInfo(L);
```

```
    // 5. Insert After
    insertAfter(L, 12, 99);
    cout << "Setelah insertAfter(12, 99): ";
    printInfo(L);

    // 6. Update
    update(L, 99, 100);
    cout << "Setelah update(99, 100): ";
    printInfo(L);

    // 7. Delete After
    delAfter(L, 12);
    cout << "Setelah delAfter(12): ";
    printInfo(L);

    // 8. Delete berdasarkan nilai
    delP(L, 8);
    cout << "Setelah delP(8): ";
    printInfo(L);

    // 9. Delete First
    delFirst(L);
    cout << "Setelah delFirst: ";
    printInfo(L);

    // 10. Delete Last
    delLast(L);
    cout << "Setelah delLast: ";
    printInfo(L);

    // 11. Menghitung jumlah node
    cout << "Jumlah node: " << nbList(L) << endl;

    // 12. Menghapus semua node
    delAll(L);
    cout << "Setelah delAll: ";
    printInfo(L);

    return 0;
}
```

Deskripsi program

Program ini mengimplementasikan Single Linked List bertipe integer (dibagi menjadi file header, implementasi, dan main) yang mendemonstrasikan operasi pencarian, insert, update, delete, hitung jumlah node, hingga penghapusan seluruh node, dengan hasil tiap langkah ditampilkan ke layar.

SOAL

1. **Soal 1**

Singlylist.h

```
#ifndef SINGLYLIST_H
#define SINGLYLIST_H

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertFirst(List &L, address P);

address findElm(List L, infotype x);
int jumlahInfo(List L);

#endif
```

Singlylist.cpp

```
#include <iostream>
#include "Singlylist.h"
using namespace std;

void createList(List &L) {
    L.first = NULL;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = NULL;
    return P;
}

void dealokasi(address &P) {
    delete P;
```

```
    P = NULL;
}

void insertFirst(List &L, address P) {
    P->next = L.first;
    L.first = P;
}

void printInfo(List L) {
    address P = L.first;

    while (P != NULL) {
        cout << P->info << " ";
        P = P->next;
    }

    cout << endl;
}

// Fungsi pencarian elemen
address findElm(List L, infotype x) {
    address P = L.first;

    while (P != NULL) {
        if (P->info == x) {
            return P;
        }
        P = P->next;
    }

    return NULL;
}

// Fungsi menghitung jumlah semua elemen
int jumlahInfo(List L) {
    address P = L.first;
    int total = 0;

    while (P != NULL) {
        total = total + P->info;
        P = P->next;
    }
    return total;
}
```

main.cpp

```
#include <iostream>
#include "Singlylist.h"
```

```
using namespace std;

int main() {
    List L;
    address P1, P2, P3, P4, P5;
    address hasil;

    createList(L);

    P1 = alokasi(2);
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    cout << "Isi linked list: ";
    printInfo(L);

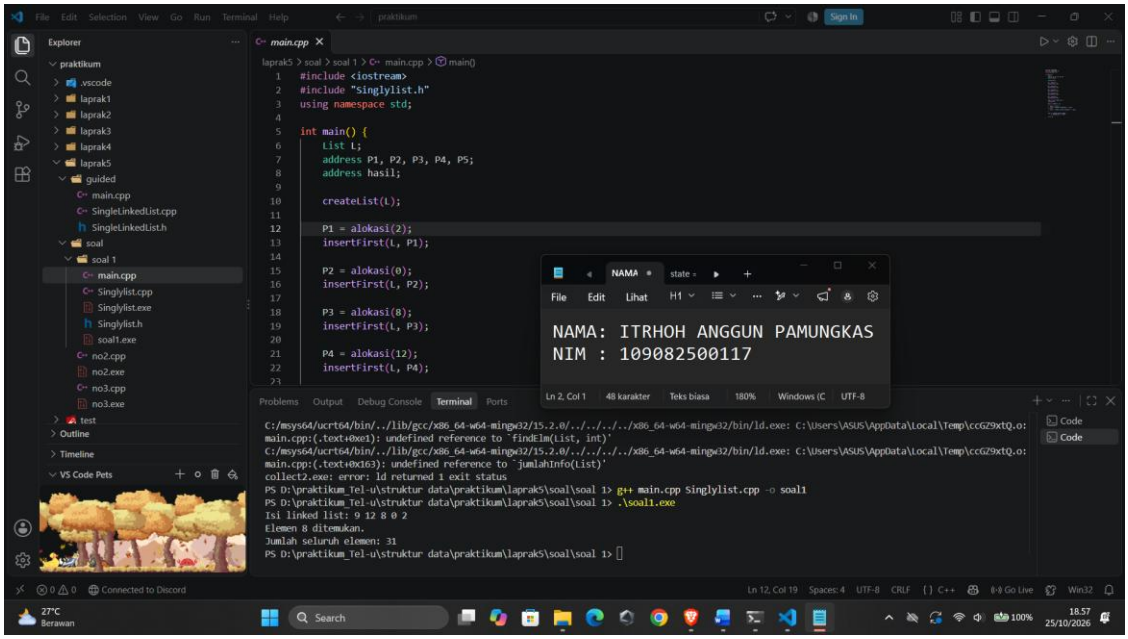
    hasil = findElm(L, 8);

    if (hasil != NULL) {
        cout << "Elemen 8 ditemukan." << endl;
    } else {
        cout << "Elemen 8 tidak ditemukan." << endl;
    }

    cout << "Jumlah seluruh elemen: "
         << jumlahInfo(L) << endl;

    return 0;
}
```

Screenshoot program



Deskripsi program

Pada program ini, membuat ADT Singly Linked List yang terdiri dari file header, file implementasi, dan file program utama. Pertama, membuat list kosong menggunakan createList(). Setelah itu, masukkan lima nilai menggunakan alokasi() dan insertFirst(). Karena menggunakan insertFirst(), nilai yang dimasukkan terakhir akan berada di posisi paling depan. Oleh karena itu, isi linked list menjadi 9 12 8 0 2.

2. Soal 2

Source Code

```
#include <iostream>
using namespace std;

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L) {
    L.first = NULL;
}
```

```
address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = NULL;
    return P;
}

void insertFirst(List &L, address P) {
    P->next = L.first;
    L.first = P;
}

void printInfo(List L) {
    address P = L.first;

    while (P != NULL) {
        cout << P->info << " ";
        P = P->next;
    }

    cout << endl;
}

address findElm(List L, infotype x) {
    address P = L.first;

    while (P != NULL) {
        if (P->info == x) {
            return P;
        }
        P = P->next;
    }

    return NULL;
}

int main() {
    List L;
    address P;
    address hasil;

    createList(L);

    P = alokasi(2);
    insertFirst(L, P);

    P = alokasi(0);
```

```
        insertFirst(L, P);

        P = alokasi(8);
        insertFirst(L, P);

        P = alokasi(12);
        insertFirst(L, P);

        P = alokasi(9);
        insertFirst(L, P);

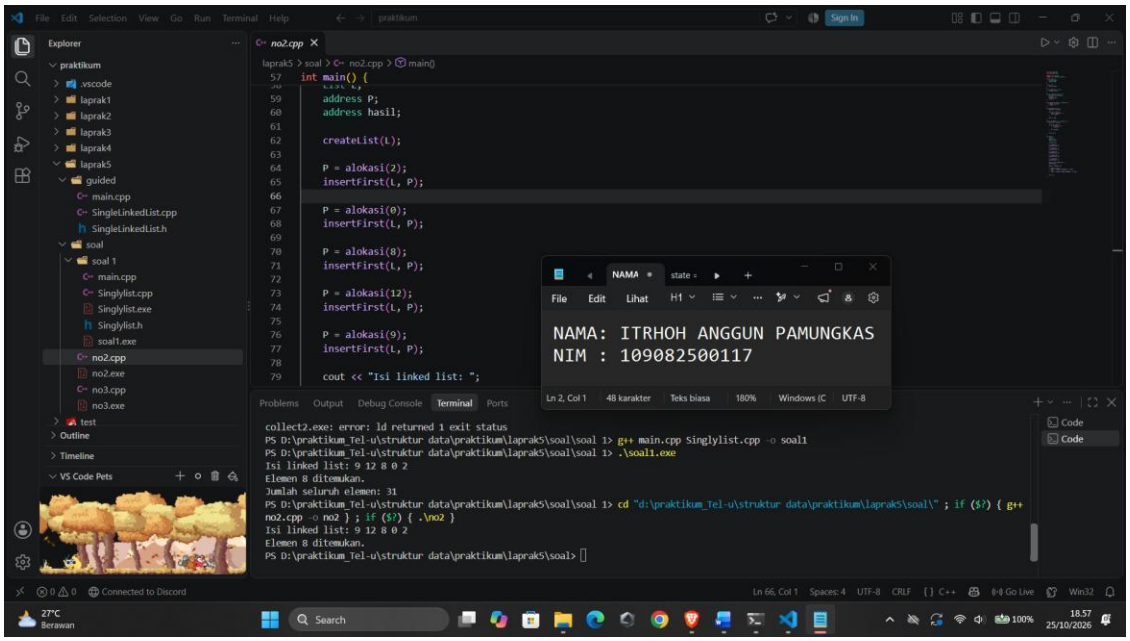
        cout << "Isi linked list: ";
        printInfo(L);

        hasil = findElm(L, 8);

        if (hasil != NULL) {
            cout << "Elemen 8 ditemukan." << endl;
        } else {
            cout << "Elemen 8 tidak ditemukan." << endl;
        }

        return 0;
    }
```

Screenshoot program



Deskripsi program

Program membuat linked list berisi lima nilai, kemudian mencari elemen yang memiliki nilai 8 menggunakan fungsi findElm(). Jika ditemukan, program menampilkan pesan bahwa elemen 8 ditemukan.

3. **Soal 3**

Source Code

```
#include <iostream>
using namespace std;

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address first;
};

void createList(List &L) {
    L.first = NULL;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = NULL;
    return P;
}

void insertFirst(List &L, address P) {
    P->next = L.first;
    L.first = P;
}

void printInfo(List L) {
    address P = L.first;

    while (P != NULL) {
        cout << P->info << " ";
        P = P->next;
    }

    cout << endl;
```

```
}

int jumlahInfo(List L) {
    address P = L.first;
    int total = 0;

    while (P != NULL) {
        total = total + P->info;
        P = P->next;
    }

    return total;
}

int main() {
    List L;
    address P;

    createList(L);

    P = alokasi(2);
    insertFirst(L, P);

    P = alokasi(0);
    insertFirst(L, P);

    P = alokasi(8);
    insertFirst(L, P);

    P = alokasi(12);
    insertFirst(L, P);

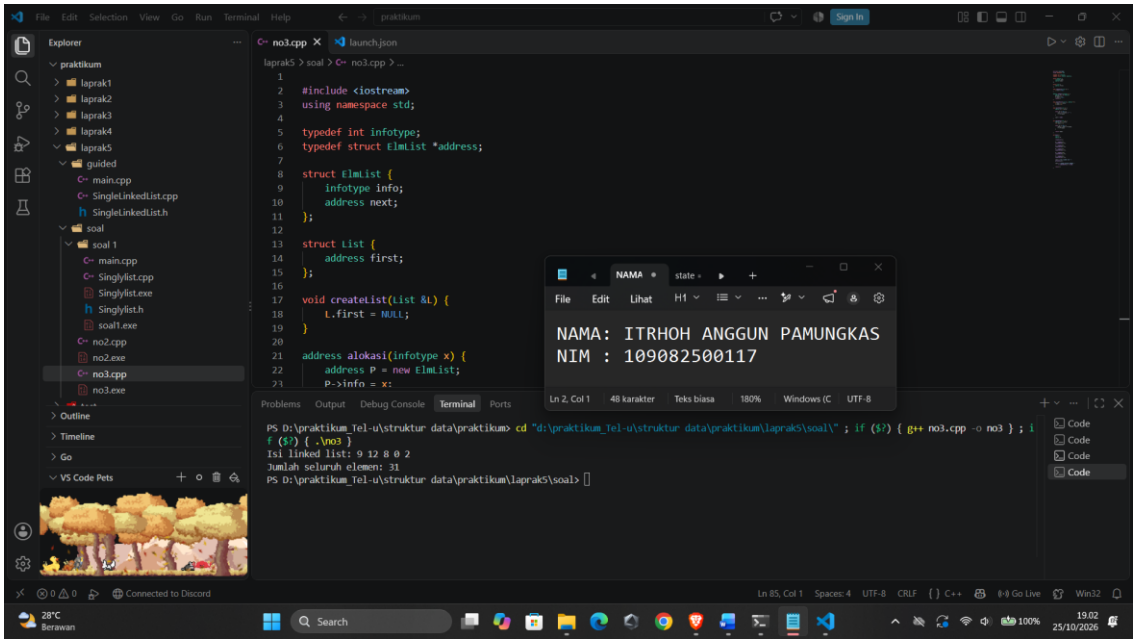
    P = alokasi(9);
    insertFirst(L, P);

    cout << "Isi linked list: ";
    printInfo(L);

    cout << "Jumlah seluruh elemen: "
         << jumlahInfo(L) << endl;

    return 0;
}
```

Screenshoot program



Deskripsi program

Program membuat linked list berisi lima nilai, kemudian menjumlahkan semua nilainya menggunakan fungsi jumlahInfo(). Hasil penjumlahan 9 + 12 + 8 + 0 + 2 adalah 31.